

A different approach to maximum clique search

Sándor Szabó

*Institute of Mathematics and Informatics
University of Pecs, Hungary
Email: sszabo7@hotmail.com*

Bogdán Zaválnij

*Alfréd Rényi Institute of Mathematics,
Hungarian Academy of Sciences
Email: bogdan@renyi.hu*

Keywords—maximum clique search; k -clique search; combinatorial optimization

Abstract—The way we tackle NP-hard problems in practical setting has experienced a major shift in recent years. Our view has become more sophisticated with the emergence of the parameterized complexity paradigm. We may distinguish subclasses inside the NP-hard complexity class. The complexity of the problems in different subclasses maybe quite different. The overall conservative estimate of the running time is replaced by a more optimistic estimate. In addition the approach of parameterized algorithms is sometimes able to deal with the more complex problems by dividing the problem into harder and a simpler parts. The easier instance at many times reduces to mere preprocessing step leaving us with only the harder part.

In this paper we single out the so-called maximum clique problem as a typical representative of the NP-hard complexity class. We propose an algorithm to solve the maximum clique problem motivated by the above ideas. Many of the available maximum clique solvers are descendants or refined versions of the Carraghan–Pardalos algorithm. (Patric Östergård’s cliquer is being as an exception.) The maximum clique problem as a maximization problem can be reduced to a series of k -clique problems as decision problems. Our main observation is that this route offers a number of advantages. The structure of a k -clique decision problem is simpler than the structure of a maximization problem. It affords additional pruning opportunities based on the available value of k . A large scale numerical experiment indicates that in many occasions the combined search space of the k -clique problems is smaller than the search space of the maximization problem. The solver we propose turns out to be rather efficient. In a number of test problems it beats the best available solvers.

I. INTRODUCTION

Let $G = (V, E)$ be a finite simple graph. The graph does not contain any double edges and the graph does not contain any loops.

The subgraph Δ of G is a clique in G if each two distinct nodes of Δ are adjacent in G . The number of nodes of Δ is called the size of the clique Δ . Instead of saying that Δ is a clique of size k we sometimes say that Δ is a k -clique in the graph G . A clique Δ is a maximum clique in G if G does not contain any clique whose size is bigger than the size of Δ .

It is an empirical fact that finding cliques in a given graph has many applications inside and outside of computer science [1], [2], [3], [4], [5], [6], [7]. We state the most commonly occurring clique search problems in a formal manner.

Problem 1: We are given a finite simple graph G . Let us determine the size of a maximum clique in G .

The size of all the maximum cliques in G is well defined common value and it is called the clique number of G . The clique number of G is denoted by $\omega(G)$. Problem 1 is referred as the maximum clique problem.

Problem 2: Given a finite simple graph G and given a positive integer k . The task is to decide if G contains a k -clique. This problem is referred as the k -clique problem.

The complexity theory of algorithms teaches us that the maximum clique optimization problem is in the NP-hard class, while the k -clique decision problem is in the NP-complete complexity class.

The Carraghan–Pardalos algorithm [8] forms the base for many exact clique search algorithms. In this sense it occupies a special position among the clique search procedures. The Carraghan–Pardalos algorithm divides a given clique search instance into smaller instances by choosing nodes into a prospective clique and repeats this process. We can view this algorithm as a good example of the well known Branch and Bound algorithm family.

In our paper we would like to introduce a different approach for the maximum clique search optimization problem. As known from the literature (see [9]) one can often build a more efficient parameterized algorithm than a general one. So we followed this path and started to build our own program, which we named *kclique*, which instead of solving the maximum clique optimization problem deals with the k -clique decision problem. Based on this program we could build a very simple and yet efficient maximum clique search program, which we will call *kclique-sequence*. We will first describe the basic algorithm techniques behind most of the state-of-the art maximum clique search programs. Second, we will detail the key features of building a k -clique search program. Third, we will shortly describe our maximum clique search program. Last, we will present results of numerical experiment. We will test our procedures using standard test graphs, and compare these results to those of the other programs. Finally, we will evaluate the results.

The program was implemented in C++ language, and we used dynamic bitsets from the Boost Template Library.

II. THE USUAL APPROACH OF MAXIMUM CLIQUE SEARCH

Most maximum clique search algorithms today based on the Carraghan–Pardalos algorithm [10], which is a classical backtracking technique. We take the nodes of the graph each by one, and reduce the graph to their neighborhood. If the reduced graph is “*not promising*” we go back, if it is “*promising*” we go forward. From another perspective we can see the basic steps from the Branch-and-Bound paradigm. For the branching step we try several different nodes, if they should be in a maximum clique. For bounding we try to prune the branches of the search tree. Algorithm 1 is the basic skeleton for algorithms based on in this paradigm. Note that $N(p)$ stands for the neighborhood of node p , that is the set of nodes connected to the node p .

Require: $G = (V, E)$

```

function MAIN
     $C^* \leftarrow \emptyset$ 
     $CP(\emptyset, V)$ 
    Print  $C^*$  as the maximum clique
end function

```

Algorithm 1 Carraghan–Pardalos algorithm

```

1: function  $CP(C, P)$ 
2:   if  $|C| > |C^*|$  then
3:      $C^* \leftarrow C$ 
4:   end if
5:   if  $|C| + |P| > |C^*|$  then
6:     for all vertex  $p \in P$  do
7:        $CP(C \cup \{p\}, P \cap N(p))$ 
8:        $P \leftarrow P \setminus \{p\}$ 
9:     end for
10:  end if
11: end function

```

We should point out some important parts in this algorithm. The bound used in the line 5 can be replaced with some sharper bound. Instead of using just the size of a subgraph, $|P|$, it is common to use the number of colors from any coloring of this subgraph – the graph spanned by the node set P –, as the number of colors used for any coloring supersedes the clique number. The combinatorial meaning of which is that we cannot take more than one node from a color class to be a part of a clique. Most often a fast coloring heuristic is used, as this coloring must be performed on each level of the search tree. Sometimes the coloring performed is using the result of the previous coloring and the proposition is to move only a few nodes between the color classes [11], [12] thus reducing the required time for coloring.

More advanced approaches may use more complex bounds. Fractional coloring [13], techniques based on linear program-

ming and semidefinite programming¹ – the Lovasz’ theta function [14], [15], [16] – are known in the literature. Some of today state-of-the-art algorithms include Maximum SAT [17] or partial MaxSAT reasoning [18].

Another important point in the algorithm is also connected to the same line. Finding a big C^* early is crucial for the bound being strong. This means that all algorithms based on the scheme depend on the early finding of a big clique, and thus use the biggest yet found clique as a bounding condition. That is possibly why the heuristics of ordering nodes by node degree in *decreasing* order plays such an important role in these algorithms. Certainly that is why parallel implementations of maximum clique search [19] find strange speed-up results – sometimes even superlinear speed-ups – in the literature, as they depend on this early finding of a big clique.

III. NUTS AND BOLTS FOR k -CLIQUE SEARCH

The main idea behind our program is the strong reduction of the size of the search tree. For both branching and bounding the choice of searching for k -clique helps us to reduce the search tree. Contrary to the heuristic approach in the Carraghan–Pardalos algorithm to base the bounding condition on an early found big clique we choose a different problem formulation. Because we should decide if a k -clique exists we can always bound by the value of k .

It is also worth noting, that the k -clique approach can help to make an efficient parallel program as well as the reader can learn in our previous publication [20].

A. Branching

One of the most convenient ways to divide a k -clique search problem into subproblems is to partition the node set V of the graph G into two subsets: A and $B = V \setminus A$. Let G_A be the graph induced by the set of nodes A . The goal of our division is to prove that $\omega(G_A) < k$, and we shall find such an A for which this problem can be easier or trivially solved. That is, if we remove the nodes of B from the graph G , the remaining graph will admit to have no k -clique and getting this information is easier than in the original problem. In the original Carraghan–Pardalos algorithm [8], this separation is based on the number of nodes. Trivially, if we choose any $(k - 1)$ nodes from V , the induced subgraph on these nodes cannot contain a k -clique. Thus, we need to remove, also eliminate, the remaining $|V| - (k - 1)$ nodes in order to solve the problem. Moreover, this elimination can be done one node at a time, producing an easier problem at each time.

More formally, let $V = \{v_1, v_2, \dots, v_n\}$. We then consider the following subsets: $V_1 = V, V_2 = V_1 \setminus v_1, V_3 = V_2 \setminus v_2, \dots, V_{n-k+1} = V_{n-k} \setminus v_{n-k}$. Clearly, solving the k -clique problem in G is equivalent to determining the existence of

¹see results of Brian Borchers at <https://oeis.org/A265032/a265032.html>

a $(k - 1)$ -clique in the induced subgraphs defined by the node sets: $N(v_1) \cap V_1, N(v_2) \cap V_2, \dots, N(v_{n-k+1}) \cap V_{n-k+1}$ where $N(v_i)$ denotes the neighbor set of v_i .

Instead of the size of the graph tighter bounds for $\omega(G)$ may be employed. The most frequently used bound is some form of greedy coloring algorithm, as in [21], [22], since computing the chromatic number is impractical for large problems. Given any legal coloring of the nodes, the graph spanned by the subset of nodes determined by any $(k - 1)$ color class cannot have a k -clique. Consequently, the separation is done in exactly the same way as before, but considering now the nodes of any partial $k - 1$ coloring of the nodes. This method was first described, with a number of minor differences, in [23].

In details, for branching we can use the value of k . It is well known, that the number of colors from any coloring gives an upper limit for the clique size. Thus if given the value of k and a coloring with c colors ($c \geq k$), then we can choose the smallest $c - (k - 1)$ color classes, and use the nodes in them for branching – as a *branching rule*. As a terminology we will call these nodes a k -clique covering node set, as introduced in [24]. The importance of this comes from the nature of the Branch and Bound algorithms. These algorithms sort out the nodes already examined, meaning that they are not taken into account in the future search. Thus if all these nodes are eliminated, then the remaining nodes can be colored with $(k - 1)$ colors, so there cannot be any k -clique present. Note, that without the value of k one cannot make this branching rule, and need to branch on all nodes.

Algorithm 2 summarizes our k clique algorithm based on this branching rule.

Algorithm 2 k clique

Require: $G = (V, E), P = V$

```

1: function  $k$ CLIQUE( $P, k$ )
2:   if  $k = 1$  then return true
3:   end if
4:   KCCNS  $\leftarrow$  construct a  $k$ -clique covering node set
5:   for all vertex  $p \in$  KCCNS do
6:     if  $k$ CLIQUE( $P \cap N(p), k - 1$ ) then return true
7:     end if
8:      $P \leftarrow P \setminus \{p\}$ 
9:   end for
10:  return false
11: end function

```

B. Efficient coloring

It is well known from the literature, that coloring of the nodes can speed-up the clique search. Two methods are widely used in the literature. One is a simple sequential greedy algorithm, where in sequence we place the nodes in the first color class it fits. The second coloring procedure

named DSatur due to Brélaz [21], which always places the least “suitable” node into a color class. The Brélaz’s algorithm usually gives a better approximation of the chromatic number than the simple sequential coloring algorithm but to carry out the coloring takes more time.

Although Dsatur gives a good coloring it is usually not optimal. So we used in addition another technique, the Iterated Coloring presented by Culberson [22]. This technique uses reordering of the color classes and using a sequential coloring several times. The result cannot be worse than the previous coloring in terms of the number of colors, but it can be better. Thus we started from a Dsatur coloring and performed iterated coloring. Our stopping criteria was if the number of colors did not decreased after 1000 iterations, and we used this method on the top of the search tree.

C. Recoloring the nodes

Some researchers proposed methods that would use the color classes from the previous level of the search tree and use a recoloring procedure to improve the actual coloring [11]. Here we present a similar approach.

As it was described by Culberson [22], the sequential greedy coloring has a special property, namely that it does not increase the number of colors if it is applied to suitable orderings of the nodes. If this procedure given a graph sequenced by color classes of a best possible coloring in which exactly so many color classes used as the chromatic number of the graph, then it will produce a coloring with the same number of color classes. If this procedure given a sequence ordered by any color classes, then it will result with a coloring at least as good as the previous one. One may see this procedure as repacking the color classes. Each node is moved forwards as far as possible in the already given color classes, and never backwards.

So during the Branch and Bound procedure, when there are less and less nodes as we go down on the search tree, we can reuse the coloring of the previous level, and use the repacking feature of the sequential greedy coloring. We sort the color classes by their size, and start a greedy sequential coloring from the biggest color class. As the k -clique covering node set is actually the $c - (k - 1)$ sets of smallest color classes, the nodes from them moved ahead to the bigger color classes. This procedure directly reduces the size of the k -clique covering node set and so the branching factor. Our tests showed, that using this method the size of the search tree is comparable with that when we would use a DSatur coloring at each level while greatly reducing the running time. We implemented this coloring procedure using bitsets, similarly to the algorithm described in [25].

D. Rearranging branching nodes

From previous results [20] on parallel clique search algorithms we concluded, that the ordering of nodes is even more important than it was thought before. It seems that the

sequence of the nodes by which we proceed in the branch has a big effect on the search tree size if pruning is present. This was shown for SAT problems [26], and could be shown for clique search problems as well. This effect is used by our algorithm, and so it reduces the search space. Note though, that this effect is different from the effect of finding a big clique early, where the sequence by *decreasing* node degrees proved useful.

We use a very basic reordering rule. We proceed with the nodes with the smallest degree in the remaining subgraph, that is, we used sequence of *increasing* node degrees. By doing this we solve first the easier problems and reduce the sizes of the later ones. Although simple and “cheap” this approach had quite a good effect on the size of the search tree.

E. Building a maximum clique search

The structure of our maximum clique problem is extremely simple. First we find an upper bound for the size of the maximum clique. Although we tried more sophisticated methods simply using the number of colors from the coloring of the graph was good enough and the fastest. Note, that we started with the Dsatur coloring and used Culberson’s Iterated Coloring scheme till the number of colors did not changed for 1000 iterations.

We set k equal to the obtained number of colors and run our k clique program with this parameter. If the result was that the graph did not contain a k -clique we decreased the value of k by one. Repeating this procedure as a sequence our program finally finds the biggest value of k for which there is a k -clique present. Thus $\omega(G) = k$. We call this program k clique-sequence, see Algorithm 3. Note, that the program calls Algorithm 2 several times.

Require: $G = (V, E)$

function MAIN

$k \leftarrow$ an upper bound by coloring

k CLIQUE-SEQ

Print k as the size of the maximum clique

end function

Algorithm 3 k clique-sequence

```

1: function  $k$ CLIQUE-SEQ
2:   FOUND  $\leftarrow$  false
3:   while  $\neg$ FOUND do
4:     FOUND  $\leftarrow$   $k$ CLIQUE( $V, k$ )
5:     if  $\neg$ FOUND then
6:        $k \leftarrow k - 1$ 
7:     end if
8:   end while
9:   return  $k$ 
10: end function

```

Our opinion was that the most time consuming part of maximum clique search is the last step, namely to prove the non-existence of a clique of size $\omega(G) + 1$. Our experiments confirmed this view in most of the cases. We also included the running time of this particular subproblem in the Table I under column “ k clique, $k = \omega(G) + 1$ ”. Also McCreesh and Prosser pointed out so in the Section 3.3 in their work [19]. This observation confirms our approach of using the k -clique search in this sequential way as opposed for example to a more complex binary search.

IV. NUMERICAL RESULTS

We performed extended measurements on a carefully selected data set of different graphs and various maximum clique search programs. As there are no challenges performed nowadays – the last well known is being the 2nd DIMACS Challenge more than 25 years ago – we used test graphs and programs that have been published about exact maximum clique search recently. Our program intends to solve previously infeasible or extremely hard problems, thus we choose appropriate problems. Most of these test problems can be solved only in minutes, if not in several hours. In the present short paper we chose 50 examples out of all together 80 graphs tested in our extended experiment. Those that were not chosen were either too easy, that is solvable under a couple of second, or too hard, that is could not be solved in 12 hours by any program including ours. We also omitted some examples that were too repetitive in kind and results.

The Table I lists graphs from various sources. The first 7 graphs are taken from the 2nd DIMACS Challenge [2]². The next 2 graphs are reformulated problems of monotonic matrices [27]³. The next 2 graphs in the table come from various error correcting code problems [28]⁴. The next 13 graphs in the table consists of Erdős–Renyi random graphs. The next 13 graphs in the table are instances from benchmark tests collected in the BHOSLIB library.⁵ The last 13 listed graphs are the so-called EVIL instances [29]⁶.

We compared our program with all together 10 well known clique search programs. We compared our program with the programs by Östergård⁷ [30] (column: cliquer), Li⁸ [31], [17] (columns: Max-CLQ10 and Max-CLQ13), Konc⁹ [32] (columns: mcqd and mcqd-dyn), Prosser¹⁰ (who implemented Tomita’s algorithm [33]) (column: MCR) and San Segundo¹¹ [34], [35], [12], [18] (columns: BBMC, BBMC-R, BBMC-L

²http://iridia.ulb.ac.be/~fmascia/maximum_clique/DIMACS-benchmark

³<http://mathworld.wolfram.com/MonotonicMatrix.html>

⁴<https://oeis.org/A265032/a265032.html>

⁵<http://www.nlsde.buaa.edu.cn/~kexu/benchmarks/graph-benchmarks.htm>

⁶<http://clique.ttk.pte.hu/evil>

⁷<http://users.aalto.fi/~pat/cliquer.html>

⁸<http://home.mis.u-picardie.fr/~cli/EnglishPage.html>

⁹<http://www.sicmm.org/konc/maxclique/>

¹⁰<http://www.dcs.gla.ac.uk/~pat/maxClique/distribution/>

¹¹https://www.biicode.com/pablodev/examples_clique

and BBMC-X). We indicated not only the running times but the size of the search tree in every second row as well. (For this purpose a small modification was made to the `cliquer` program (a counter was included), and the type of the counter in `mcqd` needed to be changed to `long long` from `int`.) Note though, that because programs may calculate the size of the search tree differently this data cannot be directly compared and only be used for rough comparison.

Column `kclique-seq` shows result from our program, $|V|$ the size of the graph and $\%$ the density of the graph.

The hardware used for performing the experiments was a Xeon E5-2670 v3 machine at 2.30GHz clock speed with 128GB of RAM, and we used a 12 hour time limit.

A. Evaluation

The fact that the clique number $\omega(G)$ of G is equal to k means the following two things. The graph G contains a k -clique and G does not contain any $(k+1)$ -clique. An algorithm to compute $\omega(G)$ verifies that G does not contain any $(k+1)$ -clique in G . Showing that G does not contain any $(k+1)$ -clique can be accomplished by an exhaustive search. Doing it efficiently requires to prune the search tree as aggressively as possible. To exhibit a k -clique in G one may apply any local search algorithm, say a tabu search. It is clear that local and exhaustive searches are based on very different strategies.

In this paper we did not comment on how local search methods should be incorporated to locate a k -clique in G . We were concerned with exhaustive search to show that G does not have any $(k+1)$ -clique. The sharp distinction between locating a k -clique and certifying the non-existence of a $(k+1)$ -clique serves only one purpose. We advance the suggestion that in the design of an exhaustive search to check the lack of a $(k+1)$ -clique the considerations related to the local search for a k -clique can be completely disregarded. (Ideas that speed up the local search should be implemented in the local search part of the over all algorithm.) In our opinion the widely used approach of the Carraghan–Pardalos algorithm mixes these two goals and tries to reach them at the same time. This may prove to be a drawback, as the two goals may contradict each other. Our approach only concentrates on one duty only, thus avoiding this contradiction. Of course only experiments can justify the usefulness of this approach.

In the paper we advance another suggestion too. It is obvious that the maximum clique problem can be reduced to a sequence of k -clique problems. In order to keep our priorities straight we suggest to focus on designing an efficient k -clique solver disregarding considerations related to maximum clique search. Concentrating on a k -clique search sets a sharply defined search target and makes possible to prune the search tree more efficiently. It allows a number of preconditioning techniques that otherwise would not be available. The utility of this approach can be assessed only by experiments. We gather evidence that our suggestions are helpful in many

occasions. We do not make the overly ambitious claim that the new algorithm dominates all the other existing clique search procedures. The fact that an algorithm out performs other algorithms on non-trivial instances is interesting enough. It is certainly an ample justification of the existence of such an algorithm.

Evaluating the numerical results the reader can see, that our simple approach of running the k -clique program with decreasing k values gives running time result that are usually comparable with, or even better than most of the best state-of-the-art programs. In 4 cases our simple approach is clearly better than more sophisticated programs that using subchromatic bounds – the `keller5` graph, the `monoton-8` graph and two `BHOSLIB` instances. It is also at second place in 10 cases – the `monoton-7`, the `latin square` and all of the `BHOSLIB` instances.¹² It is also worth noting, that our program achieves the main goal, and it reduces the search space considerably. And it is very bad on one instance, the `MANN-a45`. But one would expect exactly such a mixed behavior for a program that is based on a very different approach. Note, that the `EVIL` graph instances were designed especially “against” programs using coloring, so it is not surprising that programs using subchromatic approach from San Segundo and Li perform better than our program.

As a closing we would like to mention, that the proposed program is highly tunable, as one can use any suitable method to find the k -clique covering set. We suspect that fractional coloring or other means of finding a better bound than the chromatic number may increase the efficiency of our program.

ACKNOWLEDGMENT

This research was supported by National Research, Development and Innovation Office NKFIH Fund No. SNN-117879.

REFERENCES

- [1] I. M. Bomze, M. Budinich, P. M. Pardalos, and M. Pelillo, “The maximum clique problem,” in *Handbook of combinatorial optimization*. Springer, 1999, pp. 1–74.
- [2] J. Hasselberg, P. M. Pardalos, and G. Vairaktarakis, “Test case generators and computational results for the maximum clique problem,” *Journal of Global Optimization*, vol. 3, no. 4, pp. 463–482, 1993.
- [3] D. Kumlander, “Some practical algorithms to solve the maximal clique problem phd,” Ph.D. dissertation, Thesis, Tallin University of Technology, 2005., 2005.
- [4] C. Morgan, “A combinatorial search with dancing links,” Ph.D. dissertation, PhD. Thesis, Univ. of Warwick, 1999–2000.

¹²Although the `BHOSLIB` instances are widely used for exact maximum clique search testing we do not consider them as really good tests. This is because of the fact that in each case $\omega(G) = \chi(G)$, and it is easy to find a coloring using $\chi(G)$ colors. The good behavior of our program is clearly because of this fact.

- [5] A. Bóta and M. Krész, "A high resolution clique-based overlapping community detection algorithm for small-world networks," *Informatica*, vol. 39, no. 2, 2015.
- [6] S. Szabó and B. Zaválnij, "Reducing graph coloring to clique search," *Asia Pacific Journal of Mathematics*, vol. 3, no. 1, pp. 64–85, 2016.
- [7] S. Szabó and B. Zaválnij, "Reducing hypergraph coloring to clique search," *Discrete Applied Mathematics*, vol. in print, 2018.
- [8] R. Carraghan and P. M. Pardalos, "An exact algorithm for the maximum clique problem," *Operations Research Letters*, vol. 9, no. 6, pp. 375–382, 1990.
- [9] M. Cygan, F. V. Fomin, Ł. Kowalik, D. Lokshtanov, D. Marx, M. Pilipczuk, M. Pilipczuk, and S. Saurabh, *Parameterized algorithms*. Springer, 2015, vol. 4, no. 8.
- [10] Q. Wu and J.-K. Hao, "A review on algorithms for maximum clique problems," *European Journal of Operational Research*, vol. 242, no. 3, pp. 693–709, 2015.
- [11] A. Nikolaev, M. Batsyn, and P. San Segundo, "Reusing the same coloring in the child nodes of the search tree for the maximum clique problem," in *International Conference on Learning and Intelligent Optimization*. Springer, 2015, pp. 275–280.
- [12] P. San Segundo and C. Tapia, "Relaxed approximate coloring in exact maximum clique search," *Computers & Operations Research*, vol. 44, pp. 185–192, 2014.
- [13] E. Balas and J. Xue, "Weighted and unweighted maximum clique algorithms with upper bounds from fractional coloring," *Algorithmica*, vol. 15, no. 5, pp. 397–412, 1996.
- [14] L. Lovász, "On the shannon capacity of a graph," *IEEE Transactions on Information theory*, vol. 25, no. 1, pp. 1–7, 1979.
- [15] B. Borchers, "Csdsp, a c library for semidefinite programming," *Optimization methods and Software*, vol. 11, no. 1-4, pp. 613–623, 1999.
- [16] B. Borchers and J. G. Young, "Implementation of a primal-dual method for sdp on a shared memory parallel architecture," *Computational Optimization and Applications*, vol. 37, no. 3, pp. 355–369, 2007.
- [17] C.-M. Li, Z. Fang, and K. Xu, "Combining maxsat reasoning and incremental upper bound for the maximum clique problem," in *Tools with Artificial Intelligence (ICTAI), 2013 IEEE 25th International Conference on*. IEEE, 2013, pp. 939–946.
- [18] P. San Segundo, A. Nikolaev, and M. Batsyn, "Infra-chromatic bound for exact maximum clique search," *Computers & Operations Research*, vol. 64, pp. 293–303, 2015.
- [19] C. McCreesh and P. Prosser, "The shape of the search tree for the maximum clique problem and the implications for parallel branch and bound," *ACM Transactions on Parallel Computing*, vol. 2, no. 1, p. 8, 2015.
- [20] B. Zaválnij, "Speeding up parallel combinatorial optimization algorithms with las vegas method," in *10th International Conference on Large-Scale Scientific Computing*, Lecture Notes in Computer Science. Springer, 2015, pp. 258–266.
- [21] D. Brélaz, "New methods to color the vertices of a graph," *Communications of the ACM*, vol. 22, no. 4, pp. 251–256, 1979.
- [22] J. Culberson, "Iterated greedy graph coloring and the difficulty landscape," University of Alberta, Tech. Rep., 1992.
- [23] E. Balas and C. S. Yu, "Finding a maximum clique in an arbitrary graph," *SIAM Journal on Computing*, vol. 15, no. 4, pp. 1054–1068, 1986.
- [24] S. Szabó and B. Zaválnij, "Decomposing clique search problems into smaller instances based on node and edge colorings," *Discrete Applied Mathematics*, vol. 242, pp. 118–129, 2018.
- [25] L. Komosko, M. Batsyn, P. San Segundo, and P. M. Pardalos, "A fast greedy sequential heuristic for the vertex colouring problem based on bitwise operations," *Journal of Combinatorial Optimization*, vol. 31, no. 4, pp. 1665–1677, 2016.
- [26] M. Ouyang, "How good are branching rules in dpll?" *Discrete Applied Mathematics*, vol. 89, no. 1-3, pp. 281–286, 1998.
- [27] S. Szabó, "Monotonic matrices and clique search in graphs," *Annales Univ. Sci. Budapest., Sect. Comp*, vol. 41, pp. 307–322, 2013.
- [28] N. Sloane, "Challenge problems: Independent sets in graphs," *Information Sciences Research Center*, <http://neilsloane.com/doc/graphs.html>, 2005.
- [29] S. Szabó and B. Zaválnij, "Benchmark problems for exhaustive exact maximum clique search algorithms," *Informatica*, 2018, in print.
- [30] P. R. Östergård, "A fast algorithm for the maximum clique problem," *Discrete Applied Mathematics*, vol. 120, no. 1-3, pp. 197–207, 2002.
- [31] C. M. Li and Z. Quan, "An efficient branch-and-bound algorithm based on maxsat for the maximum clique problem," in *Proc. of AAAI*, vol. 10, 2010, pp. 128–133.
- [32] J. Konc and D. Janežic, "An improved branch and bound algorithm for the maximum clique problem," *proteins*, vol. 4, no. 5, 2007.
- [33] E. Tomita and T. Seki, "An efficient branch-and-bound algorithm for finding a maximum clique," in *Discrete mathematics and theoretical computer science*. Springer, 2003, pp. 278–289.
- [34] P. San Segundo, D. Rodríguez-Losada, and A. Jiménez, "An exact bit-parallel algorithm for the maximum clique problem," *Computers & Operations Research*, vol. 38, no. 2, pp. 571–581, 2011.
- [35] P. San Segundo, F. Matia, D. Rodríguez-Losada, and M. Hernandez, "An improved bit parallel exact maximum clique algorithm," *Optimization Letters*, vol. 7, no. 3, pp. 467–479, 2013.

	$ V $	%	$\omega(G)$	k clique, $k = \omega(G) + 1$	kclique-seq	BBMC (San Seg.)	BBMC-R (San Seg.)	BBMC-L (San Seg.)	BBMC-X (San Seg.)	Max-CLQ10 (Li)	Max-CLQ13 (Li)	mcqd (Konc)	mcqd-dyn (Konc)	MCR (Tomita)	cliquer (Östergård)
brock800-3	800	65	25	1955s	8837s	3297s	4614s	3539s	2452s	4580s	5561s	8883s	4290s	23013s	25459s
brock800-4	800	65	26	654M	2505M	234M	138M	253M	91M	218M	1545M	851M	1884M	38753M	6038s
latin-square10	900	76	90	1543s	7277s	2262s	2294s	2476s	1787s	4370s	7037s	6144s	3072s	22349s	6038s
latin-square10	900	76	90	471M	1866M	149M	88M	164M	59M	229M	1040M	564M	1915M	9554M	9554M
latin-square10	900	76	90	131s	213s	> 12h	> 12h	> 12h	> 12h	47s	> 12h	> 12h	1180s	> 12h	> 12h
latin-square10	900	76	90	7M	11M					448k			50M		
keller5	776	75	27	46s	53s	> 12h	> 12h	> 12h	> 12h	6848s	238s	> 12h	18098s	> 12h	> 12h
keller5	776	75	27	8M	9M					243M	249k		2271M		
MANN-a45	1035	99	345		> 12h	148s	67s	155s	82s	25s	123s	2384s	2058s	3692s	> 12h
MANN-a45	1035	99	345		> 12h	1M	118k	1M	118k	70k	22k	5339k	4647k	2852k	> 12h
sanr200-0.9	200	90	42	55s	141s	27s	31s	30s	21s	7s	2s	123s	30s	913s	> 12h
sanr200-0.9	200	90	42	15M	38M	4M	1370k	4M	850k	442k	8k	28M	6M	62M	121156M
sanr400-0.7	400	70	21	140s	419s	113s	135s	114s	105s	112s	117s	232s	110s	942s	3593s
sanr400-0.7	400	70	21	67M	169M	15M	9M	17M	6M	8M	477k	82M	38M	102M	5535M
monoton-7	343	79	19	3s	12s	49s	38s	42s	31s	29s	6s	151s	72s	249s	1354s
monoton-7	343	79	19	1M	5M	8M	5M	10M	5M	618k	28k	51M	20M	27M	2288M
monoton-8	512	82	23	577s	846s	17919s	23327s	17219s	15049s	4131s	1279s	> 12h	19272s	> 12h	> 12h
monoton-8	512	82	23	181M	251M	2663M	1760M	3344M	1577M	89M	6M	16125M	3386M		
1dc.256	256	88	30	2s	8s	3s	2s	3s	2s	6s	5s	15s	22s	65s	375s
2dc.1024	1024	68	16	558k	2M	382k	218k	491k	181k	156k	22k	2463k	3133k	4114k	545M
2dc.1024	1024	68	16	112s	178s	29s	36s	30s	40s	182s	199s	67s	146s	139s	3060s
2dc.1024	1024	68	16	19M	28M	2M	1509k	2M	1345k	2M	307k	8M	23M	8M	2768M
rand200-9	200	90	44	58s	204s	33s	40s	36s	30s	12s	22s	88s	1268s		> 12h
rand200-9	200	90	44	15M	52M	7M	3M	9M	2M	494k	7k	42M	14M	84M	
rand300-7	300	70	20	7s	28s	5s	8s	5s	6s	10s	5s	17s	11s	40s	95s
rand300-8	300	80	28	4M	13M	1440k	842k	1566k	583k	593k	26k	6M	4M	7M	152M
rand300-8	300	80	28	313s	984s	162s	216s	160s	142s	115s	758s	274s	2756s	29261s	
rand300-9	300	90	48	130M	379M	33M	16M	37M	11M	9M	370k	201M	72M	271M	52065M
rand300-9	300	90	48		> 12h	25333s	29220s	28071s	17574s	6645s	5894s	> 12h	9800s	> 12h	> 12h
rand300-9	300	90	48		> 12h	3288M	1256M	4083M	802M	316M	9M	2641M			
rand500-5	500	50	13	2s	7s	1s	2s	1s	1s	4s	3s	3s	7s	11s	
rand500-5	500	50	13	1M	3M	224k	143k	239k	102k	262k	18k	1110k	844k	1211k	14M
rand500-6	500	60	17	24s	86s	22s	36s	24s	20s	57s	32s	61s	39s	136s	197s
rand500-7	500	70	23	11M	35M	4M	2M	4M	1669k	3M	165k	17M	11M	20M	283M
rand500-7	500	70	23	712s	2356s	886s	1267s	662s	968s	929s	1978s	928s	6456s	16244s	
rand500-7	500	70	23	313M	898M	106M	60M	115M	40M	47M	2M	549M	227M	577M	26668M
rand800-4	800	40	12	2s	11s	1s	2s	1s	1s	7s	6s	3s	3s	7s	11s
rand800-4	800	40	12	901k	4M	137k	81k	137k	60k	377k	34k	951k	863s	1078k	14M
rand800-5	800	50	14	45s	166s	31s	51s	35s	31s	113s	107s	75s	59s	177s	297s
rand800-6	800	60	19	21M	60M	4M	3M	4M	2M	5M	491k	23M	17M	25M	348M
rand800-6	800	60	19	859s	3161s	1123s	1465s	1072s	736s	1571s	2523s	1496s	6546s	8398s	
rand800-6	800	60	19	397M	1151M	89M	54M	96M	38M	77M	5M	509M	285M	609M	12397M
rand1000-4	1000	40	12	9s	39s	7s	9s	6s	3s	20s	16s	14s	24s	24s	
rand1000-4	1000	40	12	4M	12M	734k	459k	780k	328k	1440k	64k	4M	3M	5M	27M
rand1000-5	1000	50	15	172s	679s	125s	216s	129s	152s	526s	356s	364s	265s	795s	784s
rand1000-5	1000	50	15	71M	216M	16M	10M	17M	7M	21M	1486k	87M	65M	94M	1176M
rand1000-6	1000	60	19	7519s	26398s	9858s	13019s	9552s	7164s	16558s	17093s	19683s	9833s	> 12h	> 12h
rand1000-6	1000	60	19	3258M	9441M	812M	491M	873M	335M	687M	57M	4622M	4641M		
frb50-15-1.clq	450	82	30	0s	0s	1611s	1645s	1694s	1613s	575s	0s	2735s	2541s	3673s	0s
frb50-15-2.clq	450	82	30	0	13k	256M	129M	316M	105M	25M	0k	514M	474M	288M	455k
frb50-15-2.clq	450	82	30	0s	0s	1010s	1094s	1191s	1047s	921s	0s	3329s	4155s	905s	2s
frb50-15-2.clq	450	82	30	0	30k	159M	86M	205M	68M	40M	0k	668M	878M	73M	5M
frb35-17-1.clq	595	84	35	0s	2s	> 12h	> 12h	> 12h	> 12h	> 12h	1s	> 12h	> 12h	34983s	6231s
frb35-17-1.clq	595	84	35	0	322k						0k			1723M	12630M
frb35-17-2.clq	595	84	35	0s	5s	> 12h	> 12h	> 12h	> 12h	> 12h	1s	> 12h	> 12h	> 12h	33533s
frb35-17-2.clq	595	84	35	0	1M						0k			89673M	
frb40-19-1.clq	760	86	40	0s	16s	> 12h	> 12h	> 12h	> 12h	> 12h	1s	> 12h	> 12h	11589	> 12h
frb40-19-1.clq	760	86	40	0	2M						0k			403M	
frb40-19-2.clq	760	86	40	0s	9s	> 12h	> 12h	> 12h	> 12h	> 12h	0s	> 12h	> 12h	> 12h	> 12h
frb40-19-2.clq	760	86	40	0	1M						0k				
frb45-21-1.clq	945	87	45	0s	827s	> 12h	> 12h	> 12h	> 12h	> 12h	119s	> 12h	> 12h	> 12h	> 12h
frb45-21-1.clq	945	87	45	0	111M						11k				
frb45-21-2.clq	945	87	45	0s	175s	> 12h	> 12h	> 12h	> 12h	> 12h	72s	> 12h	> 12h	> 12h	> 12h
frb45-21-2.clq	945	87	45	0	22M						5k				
frb50-23-1.clq	1150	88	50	0s	1287s	> 12h	> 12h	> 12h	> 12h	> 12h	764s	> 12h	> 12h	> 12h	> 12h
frb50-23-1.clq	1150	88	50	0	125M						46k				
frb50-23-2.clq	1150	88	50	0s	3828s	> 12h	> 12h	> 12h	> 12h	> 12h	363s	> 12h	> 12h	> 12h	> 12h
frb50-23-2.clq	1150	88	50	0	428M						19k				
frb53-24-1.clq	1272	88	53	0s	44086s	> 12h	> 12h	> 12h	> 12h	> 12h	4771s	> 12h	> 12h	> 12h	> 12h
frb53-24-1.clq	1272	88	53	0	3883M						252k				
frb53-24-2.clq	1272	88	53	0s	36514s	> 12h	> 12h	> 12h	> 12h	> 12h	190s	> 12h	> 12h	> 12h	> 12h
frb53-24-2.clq	1272	88	53	0	3309M						10k				
frb59-26-1.clq	1534	89	59	> 12h	> 12h	> 12h	> 12h	> 12h	> 12h	> 12h	> 12h	> 12h	> 12h	> 12h	> 12h
chv12x10.clq	120	92	20	1840s	1897s	4s	5s	4s	1s	1s	0s	4759s	48s	700s	0s
chv12x10.clq	120	92	20	3450M	3518M	2M	2M	3M	236k	58k	17k	17092M	87M	477M	0k
myc11x11.clq	121	93	22	233s	238s	8s	12s	7s	2s	1s	0s	1097s	85s	1081s	239s
myc5x30.clq	150	97	60	347M	350M	6M	5M	6M	241k	60k	10k	2537M	174M	845M	4860M
myc5x30.clq	150	97	60	47s	63s	1s	1s	1s	0s	0s	0s	10s	2s	47s	42042s
s3m25x6.clq	150	90	24	11M	15M	194k	43k	210k	7k	1k	0k	2M	326k	5M	235514M
s3m25x6.clq	150	90	24	258s	279s	192s	278s	195s	186s	4s	8s	64s	92s	128s	0s
chv12x15.clq	180	94	30	285M	298M	219M	176M	258M	94M	234k	127k	49M	142M	49M	0k
chv12x15.clq	180	94	30	> 12h	> 12h	26019s	34161s	26045s	7796s	1235s	184s	> 12h	> 12h	> 12h	0s
myc23x8.clq	184	90	16	623s	645s	115s	165s	112s	88s	215s	90s	1138s	1390s	> 12h	> 12h
myc23x8.clq	184	90	16	892M	906M	46M	45M	85M	24M	21M	2M	3051M	3192M		
myc11x17.clq	187	95	34	> 12h	> 12h	40109s	> 12h	33957s	5056s	2378s	2375s	> 12h	> 12h	> 12h	3159s
s3m25x8.clq	200	92	32	> 12h	> 12h	25910M	26338M	1223M	196M	38M	38M				52113M
s3m25x8.clq															